PRJ301: Java Web Application Development

Slot 15: Servlet Filter

VinhVT9@fe.edu.vn

Spring 2026

FPT University Can Tho

Learning Objectives

By the end of this slot, students will be able to:

- Explain **what a Servlet Filter is** and the problems it solves.
- Describe the **Filter architecture** and the filter chain concept.
- Implement the **Filter interface** (`init`, `doFilter`, `destroy`).
- Configure filters using `@WebFilter` annotation and `web.xml`.
- Access **initialization parameters** via `FilterConfig`.
- Control request flow: **pass through** or **block** a request.
- Build practical filters: **authentication**, **encoding**, and **logging** filters.
- Understand **filter chain ordering** with multiple filters.

FPT Education
FPT UNIVERSITY

Outline

1. What is a Filter?
2. Filter Architecture
3. The Filter Interface
4. Creating a Filter
5. Filter Configuration
6. FilterConfig
7. Filter Chain & Ordering
8. Practical Examples
9. Blocking Requests
10. Classwork

FPT Education
FPT UNIVERSITY

1. What is a Filter?

The Problem Without Filters

Consider a web application where many servlets need the same logic:

- Check if the user is **logged in** before accessing any protected page.
- Set the **character encoding** to UTF-8 for every request.
- **Log** every incoming request with timestamp and IP address.

The Bad Approach
Copy-pasting the same code into every Servlet is:
— error-prone — hard to maintain — violates DRY principle

Solution: Use a **Servlet Filter** — intercept requests once, apply logic centrally.

FPT Education
FPT UNIVERSITY

What is a Servlet Filter?

- A **Filter** is a reusable Java component that intercepts HTTP requests and responses *before* they reach a Servlet (or JSP).
- Filters are part of the **Jakarta Servlet API** (`jakarta.servlet.Filter`).
- A filter can **inspect**, **modify**, or **block** requests and responses.
- Multiple filters can be chained together — each filter decides whether to pass the request to the next one.

Key Interface

`jakarta.servlet.Filter` — three methods:

`init(FilterConfig)` `doFilter(req, resp, chain)` `destroy()`

FPT Education
FPT UNIVERSITY

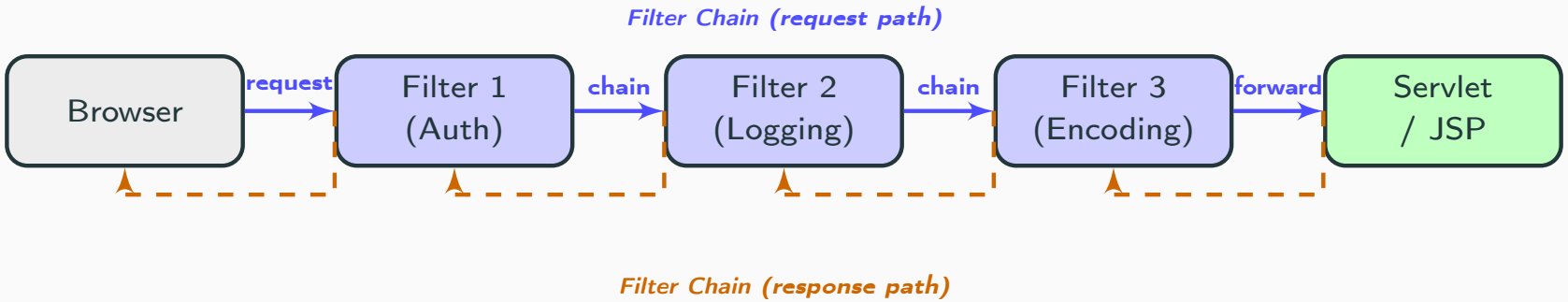
Common Filter Use Cases

Use Case	Description
Authentication	Block requests from unauthenticated users
Logging / Auditing	Track which user accessed which URL and when
Character Encoding	Set UTF-8 encoding on every request
Data Compression	Compress response output (GZIP)
Image Conversion	Scale or transform images on the fly
Localization	Set locale based on request headers
XSL/T Transformation	Transform XML responses to HTML
CORS Headers	Add cross-origin response headers

FPT Education
FPT UNIVERSITY

2. Filter Architecture

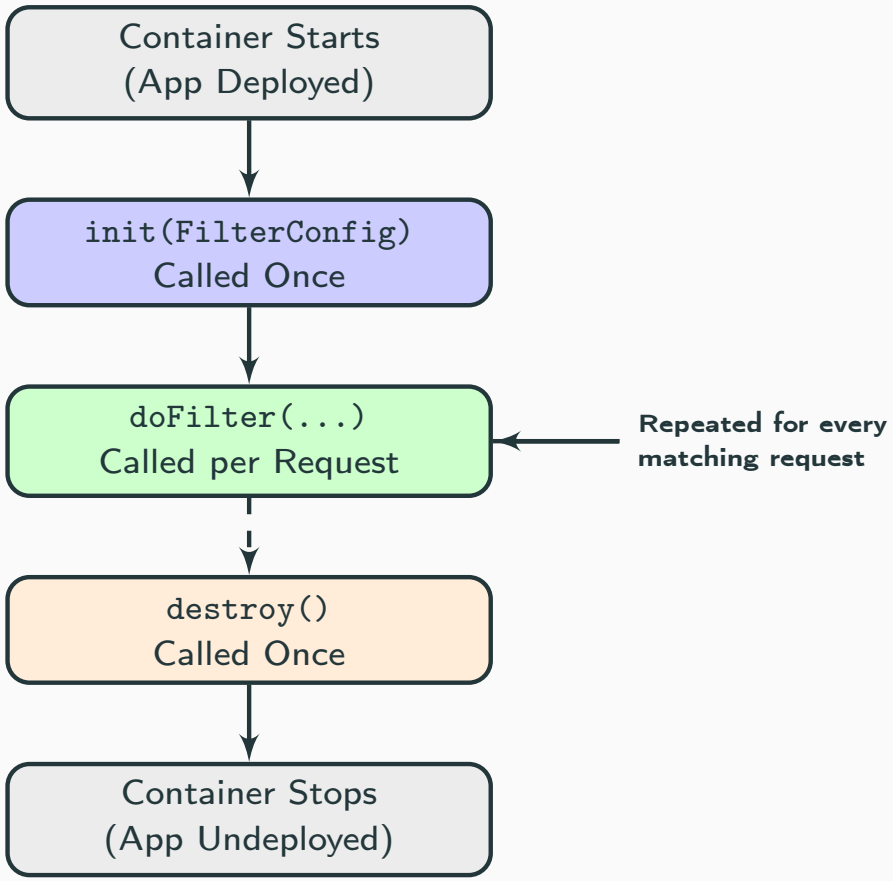
Filter Architecture: The Filter Chain



Block: If Filter 1 rejects the request, it does NOT call `chain.doFilter()` and the Servlet is never reached.

FPT Education
FPT UNIVERSITY

Filter Lifecycle



FPT Education
FPT UNIVERSITY

3. The Filter Interface

The Filter Interface

```java
package jakarta.servlet;

public interface Filter {

    // Called once when the filter is initialized
    default void init(FilterConfig filterConfig)
        throws ServletException { }

    // Called for every matching request
    void doFilter(
        ServletRequest request,
        ServletResponse response,
        FilterChain chain)
        throws IOException, ServletException;

    // Called once when the filter is destroyed
    default void destroy() { }
}
```

Listing 1: jakarta.servlet.Filter

FPT Education
FPT UNIVERSITY

The `FilterChain` Interface

```java
package jakarta.servlet;

public interface FilterChain {

    /**
     * Passes the request and response to the
     * next filter, or to the target Servlet/JSP
     * if this is the last filter in the chain.
     */
    void doFilter(
        ServletRequest request,
        ServletResponse response)
        throws IOException, ServletException;
}
```

Listing 2: jakarta.servlet.FilterChain

- Call `chain.doFilter(req, resp)` to **continue** the chain.
- Do NOT call it to **block** the request (e.g., send redirect instead).

FPT Education
FPT UNIVERSITY

4. Creating a Filter

Minimal Filter Implementation

```java
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebFilter;
import java.io.IOException;

@WebFilter("/*")
public class MyFilter implements Filter {

    @Override
    public void init(FilterConfig config)
            throws ServletException {
        // one-time initialization
    }

    @Override
    public void doFilter(ServletRequest req,
            ServletResponse res, FilterChain chain)
            throws IOException, ServletException {
        // pre-processing here

        chain.doFilter(req, res); // pass through

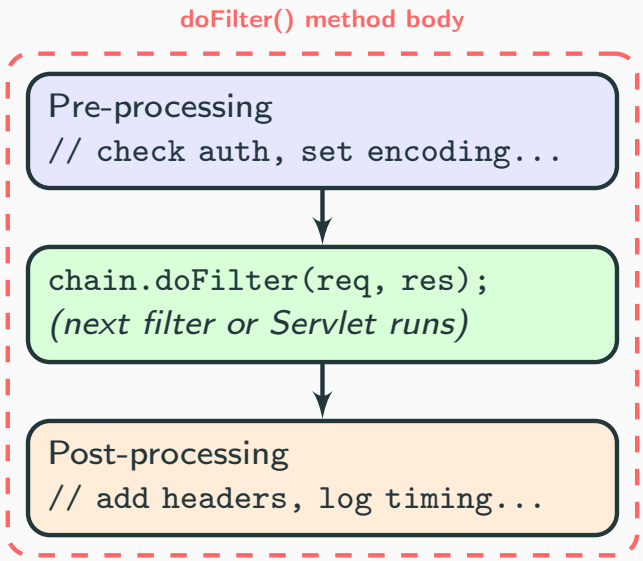
        // post-processing here
    }

    @Override
    public void destroy() {
        // cleanup resources
    }
}
```

Listing 3: MyFilter.java — basic skeleton

FPT Education
FPT UNIVERSITY

Filter Request/Response Flow



Important

Code **before** `chain.doFilter()` runs on the **request**.

Code **after** `chain.doFilter()` runs on the **response**.

FPT Education
FPT UNIVERSITY

5. Filter Configuration

Option A: `@WebFilter` Annotation

```java
import jakarta.servlet.annotation.WebFilter;

// Map to all URLs
@WebFilter("/*")
public class EncodingFilter implements Filter { ... }

// Map to specific path
@WebFilter("/admin/*")
public class AdminFilter implements Filter { ... }

// Map to specific servlet name
@WebFilter(servletNames = {"MyServlet"})
public class AuditFilter implements Filter { ... }

// Multiple URL patterns
@WebFilter(urlPatterns = {"/api/*", "/user/*"})
public class ApiFilter implements Filter { ... }
```

Listing 4: Annotation-based configuration

FPT Education
FPT UNIVERSITY

Option B: `web.xml` Configuration

```
<filter>
    <filter-name>AuthFilter</filter-name>
    <filter-class>
        fptu.prj301.filter.AuthFilter
    </filter-class>
    <init-param>
        <param-name>loginPage</param-name>
        <param-value>/login.jsp</param-value>
    </init-param>
</filter>

<filter-mapping>
    <filter-name>AuthFilter</filter-name>
    <url-pattern>/protected/*</url-pattern>
</filter-mapping>
```

Listing 5: web.xml filter registration

FPT Education
FPT UNIVERSITY

URL Patterns in Filter Mapping

Pattern	Matches
`/*`	Every request in the application
`/admin/*`	All paths under `/admin/`
`*.jsp`	All JSP files
`/api/users`	Exact path only
`/secure/*`	Paths under `/secure/`

Note

URL patterns for filters follow the same rules as Servlet URL patterns. The container matches patterns in the order they are declared in `web.xml`.

FPT Education
FPT UNIVERSITY

6. FilterConfig

Accessing Initialization Parameters

```java
@WebFilter(
    urlPatterns = "/*",
    initParams = {
        @WebInitParam(
            name  = "excludedPaths",
            value = "/login,/register,/css"
        )
    }
)
public class AuthFilter implements Filter {

    private String[] excludedPaths;

    @Override
    public void init(FilterConfig config)
            throws ServletException {
        String param = config.getInitParameter(
            "excludedPaths");
        excludedPaths = param.split(",");
    }
    // ...
}
```

Listing 6: Using FilterConfig

FPT Education
FPT UNIVERSITY

FilterConfig Methods

```java
public interface FilterConfig {

    // Name declared in @WebFilter or web.xml
    String getFilterName();

    // Access the servlet context
    ServletContext getServletContext();

    // Get single init parameter
    String getInitParameter(String name);

    // Get all init parameter names
    Enumeration<String> getInitParameterNames();
}
```

Listing 7: FilterConfig API

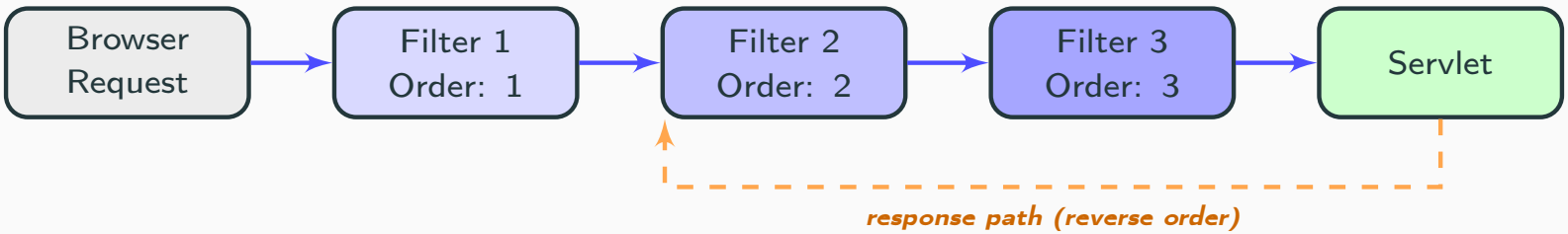
- `getServletContext()` gives access to the application scope (same as `application` in JSP).
- Store any parsed init parameters as **instance fields** in the `init()` method.

FPT Education
FPT UNIVERSITY

7. Filter Chain & Ordering

Filter Chain: Order of Execution

When multiple filters match a URL, they execute in a defined order:



Ordering Rules

- `web.xml`: Order of `<filter-mapping>` elements.
- `@WebFilter`: Order is **not guaranteed** — use `web.xml` when order matters.

FPT Education
FPT UNIVERSITY

Controlling Order with `web.xml`

```
<!-- Filter 1 runs first -->
<filter-mapping>
    <filter-name>EncodingFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>

<!-- Filter 2 runs second -->
<filter-mapping>
    <filter-name>AuthFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>

<!-- Filter 3 runs third -->
<filter-mapping>
    <filter-name>LoggingFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

Listing 8: Explicit ordering via web.xml

FPT Education
FPT UNIVERSITY

8. Practical Examples

Example 1: Character Encoding Filter

```java
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebFilter;
import java.io.IOException;

@WebFilter("/*")
public class EncodingFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest req,
            ServletResponse res,
            FilterChain chain)
            throws IOException, ServletException {

        // Set encoding BEFORE reading parameters
        req.setCharacterEncoding("UTF-8");
        res.setCharacterEncoding("UTF-8");
        res.setContentType(
            "text/html; charset=UTF-8");

        chain.doFilter(req, res);
    }
}
```

Listing 9: EncodingFilter.java

FPT Education
FPT UNIVERSITY

Example 2: Authentication Filter

```java
@WebFilter("/protected/*")
public class AuthFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest  req =
            (HttpServletRequest)  request;
        HttpServletResponse res =
            (HttpServletResponse) response;

        HttpSession session = req.getSession(false);
        boolean loggedIn = (session != null)
            && (session.getAttribute("user") != null);

        if (loggedIn) {
            chain.doFilter(request, response);
        } else {
            res.sendRedirect(
                req.getContextPath() + "/login.jsp");
        }
    }
}
```

Listing 10: AuthFilter.java

FPT Education
FPT UNIVERSITY

Example 3: Logging Filter

```java
@WebFilter("/*")
public class LoggingFilter implements Filter {

    private ServletContext context;

    @Override
    public void init(FilterConfig config)
            throws ServletException {
        context = config.getServletContext();
    }

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest req =
            (HttpServletRequest) request;
        long start = System.currentTimeMillis();

        context.log(req.getRemoteHost()
            + " -> " + req.getRequestURI());

        chain.doFilter(request, response);

        long elapsed =
            System.currentTimeMillis() - start;
        context.log("  completed in "
            + elapsed + "ms");
    }
}
```

Listing 11: LoggingFilter.java

FPT Education
FPT UNIVERSITY

9. Blocking Requests

Blocking vs. Passing Through

```java
@Override
public void doFilter(
        ServletRequest request,
        ServletResponse response,
        FilterChain chain)
        throws IOException, ServletException {

    HttpServletRequest  req =
        (HttpServletRequest)  request;
    HttpServletResponse res =
        (HttpServletResponse) response;

    if (isAllowed(req)) {
        // PASS: let the request continue
        chain.doFilter(request, response);
    } else {
        // BLOCK: respond directly
        res.sendRedirect(
            req.getContextPath() + "/error.jsp");
        // Do NOT call chain.doFilter() here
    }
}
```

Listing 12: Decision logic in doFilter()

FPT Education
FPT UNIVERSITY

Forward vs. Redirect in Filters

RequestDispatcher.forward()	response.sendRedirect()
Server-side	Client-side (2 HTTP round-trips)
URL in browser does NOT change	URL in browser changes
Request attributes preserved	Request attributes lost
Faster (one request)	Slightly slower
Cannot redirect outside app	Can redirect anywhere

In Filters

Use `sendRedirect()` when blocking (e.g., send to login page).

Use `forward()` when you want to change the target internally.

FPT Education
FPT UNIVERSITY

Using `RequestDispatcher.forward()` in a Filter

```java
@Override
public void doFilter(
        ServletRequest request,
        ServletResponse response,
        FilterChain chain)
        throws IOException, ServletException {

    HttpServletRequest req =
        (HttpServletRequest) request;

    if (!isAuthenticated(req)) {
        // Forward keeps original URL unchanged
        req.setAttribute("error",
            "Please log in first.");
        req.getRequestDispatcher("/login.jsp")
           .forward(request, response);
        // Return without calling chain
        return;
    }
    chain.doFilter(request, response);
}
```

Listing 13: Forward instead of redirect

FPT Education
FPT UNIVERSITY

10. Classwork

Classwork 5: Filter Tasks

Task A: Encoding Filter
Create `EncodingFilter` that:

1. Maps to `/*` using `@WebFilter`.
2. Sets request and response character encoding to UTF-8.
3. Test with a form that submits Vietnamese text.

Task B: Authentication Filter
Build a login system with a filter that:

1. Protects all URLs under `/protected/*`.
2. Reads `user` from the `HttpSession`.
3. Redirects to `/login.jsp` if not logged in.
4. Allows whitelisted paths (login, register, CSS).

FPT Education
FPT UNIVERSITY

Classwork 5: Discussion Questions

1. What is the difference between calling `chain.doFilter()` and **not** calling it inside `doFilter()`?
2. Why is `@WebFilter` order **not guaranteed** while `web.xml` ordering is? When should you prefer `web.xml`?
3. Compare **filter-based authentication** with putting session-check code inside each Servlet. What are the trade-offs?
4. A filter runs code **after** `chain.doFilter()`. Give a practical example of useful post-processing in a filter.
5. What happens if two filters both map to `/*` and the first filter does NOT call `chain.doFilter()`?

FPT Education
FPT UNIVERSITY

Summary: Slot 15 — Servlet Filter

- **What is a Filter?** A component that intercepts requests/responses — ideal for cross-cutting concerns.
- **Filter Interface:** `init()`, `doFilter()`, `destroy()` in `jakarta.servlet.Filter`.
- **FilterChain:** Call `chain.doFilter()` to pass through; omit it to block.
- **Configuration:** `@WebFilter` annotation or `web.xml` `<filter-mapping>`.
- **FilterConfig:** Access init parameters via `getInitParameter()`.
- **Ordering:** Use `web.xml` when filter execution order matters.
- **Common patterns:** Auth filter, encoding filter, logging filter.
- **Pre vs. Post:** Code before `chain.doFilter()` = request path; after = response path.

Up Next
Slot 16: Session Management — tracking users across requests.

FPT Education
FPT UNIVERSITY

Questions?

FPT Education
FPT UNIVERSITY